

Full Application – Test, Fix & Deploy

You are responsible for taking the current application from its existing state to a fully tested, production-ready, deployed application.

Do NOT just explain what I need to do. Perform the work yourself wherever possible. Inspect the existing project first and understand its current architecture, configuration, environment variables, frontend, backend, database, authentication, APIs, and deployment setup.

The application should be considered complete only when the deployed frontend and backend communicate correctly and the major user flows work end-to-end.

1. FIRST: INSPECT THE ENTIRE PROJECT

Before making changes:

- Inspect the complete folder structure.
- Identify frontend and backend.
- Identify the technologies currently being used.
- Do NOT unnecessarily migrate or rewrite the existing application.
- Preserve the existing architecture unless a change is required for deployment or correctness.
- Inspect package.json, environment configuration, API configuration, database configuration, authentication, routing, CORS, build scripts, TypeScript/Vite configuration, Prisma/Mongoose configuration, deployment configuration, and Git configuration.
- Identify all hardcoded localhost URLs, ports, credentials, API endpoints, and development-only configurations.

Create a checklist of everything that must be fixed before deployment.

2. RUN LOCAL TESTS BEFORE DEPLOYMENT

Run the appropriate commands for the project.

Frontend:

- Install dependencies if necessary.
- Run TypeScript/build checks.
- Run linting if configured.
- Run the production build.

Backend:

- Install dependencies if necessary.
- Run TypeScript/build checks if applicable.
- Start the backend in production mode.

- Verify that the server starts without errors.

Database:

- Verify database connection.
- Verify schema and migrations.
- Verify seed data if required.
- Verify required tables/collections and indexes.

If a command fails:

- 1. Identify the root cause.**
- 2. Fix the issue.**
- 3. Run the command again.**
- 4. Continue until relevant checks pass.**

Do not simply report an error and stop.

3. ENVIRONMENT VARIABLES

Audit every environment variable used by the application.

Separate frontend-safe variables from backend secrets.

Check variables such as:

- Database URL
- JWT/authentication secrets
- API keys
- Frontend API URL
- Backend URL
- CORS origins
- OAuth configuration if present
- AI/API integration keys

Remove accidentally hardcoded secrets from source code.

NEVER commit real passwords, API keys, JWT secrets, or database credentials to Git.

Create/update .env.example files containing variable names but NOT real secret values.

4. FRONTEND PRODUCTION CONFIGURATION

Make the frontend production-ready.

Check:

- API base URL
- Environment variable usage

- React Router configuration
- SPA routing
- Production build
- Asset paths
- Direct route refreshes
- Authentication persistence
- Logout
- Protected routes
- Error states
- Loading states
- Mobile responsiveness
- Console errors
- Network/API errors

The deployed frontend MUST NOT attempt to communicate with localhost or 127.0.0.1 unless explicitly required for local development.
Every API request must point to the deployed backend.

5. BACKEND PRODUCTION CONFIGURATION

Make the backend production-ready.

Check:

- Production start command
- PORT environment variable
- CORS
- Database connection
- Authentication
- JWT handling
- API routes
- Error handling
- Request validation
- Static files if applicable

- Health-check endpoint

If a health endpoint does not already exist, create an appropriate GET /health endpoint that returns a simple successful response indicating the backend is running. Do not expose sensitive information.

6. CORS

Configure CORS correctly for production.

The backend should allow requests from the actual deployed frontend domain.

Do NOT blindly use origin: "*" when authentication/cookies/security requirements make that inappropriate.

Ensure frontend → backend API requests work in production.

7. DATABASE

Verify production database configuration.

Make sure:

- Production database URL is configured.
- Schema is synchronized.
- Required migrations are applied.
- Required seed/admin/demo data exists if needed.
- Backend can connect to production database.

If Prisma is used, verify Prisma generation and migrations.

If MongoDB/Mongoose is used, verify connection and required collections/data.

Do not delete existing production data unless explicitly instructed.

8. AUTHENTICATION TESTING

Test the complete authentication flow.

Login:

- Valid credentials → successful login.
- Invalid credentials → proper error.
- Empty fields → validation error.
- Token/session is stored correctly.

Protected pages:

- Authenticated user → access allowed.
- Unauthenticated user → redirected appropriately.

Logout:

- Session/token is removed.
- Protected pages become inaccessible.

If the application has roles such as Admin, Employee, User, Manager, etc., test

that each role sees the correct dashboard and permissions.

9. END-TO-END APPLICATION TESTING

Do not stop at checking whether the website opens.

For every major feature:

- 1. Open the UI.**
- 2. Perform the action.**
- 3. Verify the frontend request.**
- 4. Verify backend response.**
- 5. Verify database changes where applicable.**
- 6. Verify the updated UI.**

Test:

- Login
- Dashboard
- Navigation
- CRUD operations
- Forms
- Search
- Filters
- Profile/settings
- Role switching if applicable
- Notifications
- AI features if implemented
- Language switching if implemented
- Logout
- Error handling

Use realistic test data.

10. API TESTING

Test all important API endpoints.

For each endpoint verify:

- Correct HTTP method
- Correct URL
- Authentication requirements
- Request body

- Response status
- Response structure
- Error handling
- Database interaction

Check for unexpected:

- 404 errors
- 401 errors
- 403 errors
- 400 validation errors
- 500 server errors

Fix unexpected errors.

11. UI / UX TESTING

Test:

- Desktop
- Tablet
- Mobile

Check:

- Navbar alignment
- Sidebar
- Cards
- Tables
- Forms
- Buttons
- Modals
- Dropdowns
- Charts
- Text overflow
- Responsive layouts
- Scrolling

- Touch interactions

There should be no:

- Broken layouts
- Overlapping elements
- Unnecessary scrollbars
- Placeholder content
- Broken links
- Broken images
- Visible development/debug UI

Also check browser console for errors and warnings.

12. DEPLOYMENT

Deploy using the deployment providers already configured for the project.

If configured for Vercel, deploy the frontend appropriately.

If configured for Render, deploy the backend appropriately.

If configured for MongoDB Atlas, verify the database.

If configured for PostgreSQL/Neon/Supabase/etc., verify the database.

Do NOT unnecessarily change providers.

Verify:

- Frontend production build succeeds and deployment is live.
- Backend production server starts successfully and is reachable.
- Database connection works.
- Frontend successfully communicates with backend.

13. PRODUCTION SMOKE TEST

After deployment, perform a complete smoke test against the LIVE application.

Do NOT consider deployment successful merely because the deployment provider says "Build successful."

Verify:

- 1. Open live frontend.**
- 2. Login.**
- 3. Confirm authentication works.**
- 4. Open dashboard.**
- 5. Navigate through major pages.**
- 6. Perform important CRUD operations.**
- 7. Confirm API calls succeed.**
- 8. Confirm database updates.**
- 9. Test logout.**

- 10. Test protected routes.**
- 11. Test different roles if applicable.**
- 12. Check browser console.**
- 13. Check backend logs.**
- 14. Check network requests.**
- 15. Confirm there are no localhost API calls.**
- 14. SECURITY CHECK**

Check for:

- Hardcoded secrets
- Exposed API keys
- Exposed database credentials
- Unsafe CORS
- Debug endpoints
- Development credentials
- Sensitive information in frontend bundles
- Excessive error information
- Insecure authentication configuration

Do not expose secrets in the final response.

If an existing credential should not be committed, remove it from the code/configuration and tell me that the credential should be rotated.

15. PERFORMANCE / PRODUCTION CHECK

Check for obvious production issues:

- Extremely large assets
- Unnecessary API requests
- Infinite API loops
- Excessive database queries
- Missing loading states
- Missing error states
- React rendering issues
- Unhandled promises
- Memory/resource issues
- Production console errors

Do not over-engineer optimization. Fix clear issues that can affect the demo or production reliability.

16. FIX — DON'T JUST REPORT

If you encounter an error during testing or deployment:

DO NOT STOP AND ASK ME TO FIX IT.

Instead:

- 1. Investigate the error.**
- 2. Determine the root cause.**
- 3. Make the required code/configuration change.**
- 4. Re-run the failed test.**
- 5. Continue testing.**

Only ask me for input if the issue genuinely requires a decision, credential, permission, or information that cannot be obtained from the project.

17. FINAL VERIFICATION

Before declaring the application complete, verify:

- ☐ Frontend builds successfully
- ☐ Backend builds successfully
- ☐ Database connects successfully
- ☐ Production environment variables configured
- ☐ No localhost API URLs in production
- ☐ CORS works
- ☐ Authentication works
- ☐ Protected routes work
- ☐ Role-based access works
- ☐ Major APIs work
- ☐ Major CRUD operations work
- ☐ Dashboard works
- ☐ AI features work, if present
- ☐ Multilingual features work, if present
- ☐ Mobile UI works
- ☐ Desktop UI works
- ☐ No critical browser console errors
- ☐ No critical backend errors

- ☐ Production deployment succeeds
- ☐ Live frontend loads
- ☐ Live backend responds
- ☐ Live frontend ↔ backend communication works
- ☐ Production database operations work
- ☐ Final smoke test passes

18. FINAL RESPONSE

When everything is complete, give me a concise deployment report containing:

Deployment Status

- Frontend: PASS/FAIL
- Backend: PASS/FAIL
- Database: PASS/FAIL
- Frontend ↔ Backend: PASS/FAIL
- Authentication: PASS/FAIL
- End-to-end testing: PASS/FAIL

URLs

- Actual deployed frontend URL
- Actual deployed backend URL

Tests Performed

- List the important tests actually executed.

Problems Fixed

- List important issues discovered and fixed.

Remaining Issues

- Only list genuine remaining issues.

Do NOT claim something passed unless you actually tested it.

The required workflow is:

BUILD → TEST → FIX → RE-TEST → DEPLOY → LIVE TEST → VERIFY

Do not stop at “deployment successful.” The final requirement is a working production application that has been tested end-to-end.